ÉNONCÉ DU PROJET FINAL :

ENGINE DE TRAITEMENT ET D'ANALYSE DE FLUX DE DONNÉES SÉCURISÉ EN TEMPS RÉEL

Niveau : ING3
Format : En trinôme (3 étudiants)
Langage : Python 3.10+
Durée : 3 à 4 semaines

OBJECTIF DU PROJET

L'objectif de ce projet est de concevoir et développer une plateforme complète en ligne de commande (CLI) d'ingestion, de filtrage, de transformation et d'analyse de données système/réseau complexes issues de journaux de logs volumineux.

L'application doit être capable de traiter un volume très important de données en mémoire de manière optimisée sans charger l'intégralité des fichiers en RAM. Vous devez appliquer une architecture modulaire en respectant les bonnes pratiques d'isolation d'environnement et de structuration du code.

ARCHITECTURE & LIVRABLES OBLIGATOIRES

1. Gestion de l'Environnement Virtuel & Dépendances

- Le projet **doit** fonctionner dans un environnement virtuel isolé (venv ou conda).
- Vous devez fournir un fichier requirements.txt généré via pip freeze (ou un fichier environment.yml pour Conda) permettant de reclasser et réinstaller fidèlement l'environnement.
- Aucune bibliothèque globale ou non listée dans le fichier d'environnement ne sera tolérée.

2. Structure des Fichiers & Scripts

- Le code doit être structuré sous forme de package Python (scripts .py propres et exécutables).
- Les notebooks Jupyter (.ipynb) ne sont autorisés **que** pour la partie expérimentation / prototypage et démonstration graphique finale.
- Un point d'entrée principal (main.py) pilotera l'ensemble du pipeline.

CAHIER DES CHARGES TECHNIQUE (PARTIES DU PROJET)

Module 1 : Ingestion de Données & Générateurs Mémoire

Le système doit pouvoir ingérer des fichiers de logs volumineux ou des flux textuels simulés sans jamais faire s'effondrer la RAM.

1. **Générateur de Logs** :
    - Implémentez un générateur Python basé sur yield pour lire un fichier de logs ligne par ligne à la demande.
    - Développez un sous-générateur secondaire qui lit plusieurs sources/fichiers de logs successifs en utilisant l'instruction yield from.

2. **Parsing & Modélisation des Données** :

    - Chaque ligne doit être parsée et structurée sous forme de **Dictionnaire Python**.
    - Les données doivent inclure des types variés : horodatage, niveau de sévérité (INFO, WARNING, ERROR, CRITICAL), adresse IP source, identifiant utilisateur, payload JSON ou chaîne brute.
    - Gestion de structures immutables : Utilisez des **Tuples** pour représenter les métadonnées fixes (ex: (IP, Port, Protocole)).

Module 2 : Pipeline de Transformation & Programmation Fonctionnelle

Une fois les logs sous forme d'itérateurs, appliquez une suite de transformations en chaîne.

1. **Filtrage Intelligent** :
    - Utilisez la fonction filter() combinée à des fonctions anonymes lambda pour exclure les requêtes invalides ou nettoyer les entrées vides (filter(None, ...) ou prédicats personnalisés).
2. **Transformations & Normalisation** :
    - Utilisez la fonction map() pour appliquer une fonction de normalisation (nettoyage des chaînes, conversion de la casse, reformatage des f-strings) sur chaque enregistrement.
3. **Analyse de Structure** :
    - Remplacer les constructions complexes if-elif-else par le **Structural Pattern Matching (**match-case**)** pour router les logs selon leur format :
        - Chaîne brute
        - Dictionnaire (ex: {'level': 'ERROR', 'code': code})
        - Tuples de connexion

Module 3 : Agrégation & Traitement de Données Avancé

1. **Gestion des Ensembles & Unicités** :

- Identifiez l'ensemble des adresses IP uniques ayant généré des erreurs en utilisant des set.
- Effectuez des opérations d'ensembles mathématiques (Unions |, Intersections &, Différences -, Différences symétriques ^) entre la liste des IP blacklistées (représentée sous forme de frozenset immuable) et les IP actives.

2. **AccumulationDynamique** :
   - Utilisez la classe defaultdict du module collections pour compter le nombre d'occurrences par catégorie de log ou par utilisateur sans risquer de lever d'exceptions KeyError.

3. **Contrôle de Flux & Boucles Complexe** :
   - Développez un algorithme de détection d'attaques par force brute (ex: X échecs de connexion consécutifs) en combinant des boucles while et for.
   - Exploitez impérativement les instructions break, continue, ainsi que la clause else **associée aux boucles** pour valider la réussite complète d'un cycle de détection.

Module 4 : Moteur de Règles Modulaire & Introspection Dynamique

Flexible Rule Engine :
- Créez une fonction maître apply_security_rules(*args, **kwargs) capable de recevoir un nombre variable de fonctions de sécurité (passées comme arguments de type *args) ainsi que des paramètres de configuration dynamiques (passés via **kwargs).
- Rappel : Les fonctions doivent être manipulées comme des objets de première classe (passées en argument à d'autres fonctions).

2. **Introspection & Context Audit** :
   - Implémentez un module d'audit système qui inspecte l'état d'exécution à l'aide des fonctions globals() et locals() afin d'exporter un rapport sur les variables de portée actuellement chargées en mémoire pour des raisons d'audit et de débogage.

BARÈME ET CRITÈRES D'ÉVALUATION (20 POINTS)

Critère	Description	Barème
Respect de l'Environnement & Livrables	Prescriptions venv/conda respectées, fichier requirements.txt fonctionnel, distinction propre .py / .ipynb.	**2 pts**
Gestion Mémoire & Generateurs	Utilisation appropriée de yield et yield from sans charger tout le jeu de données en RAM.	**4 pts**

Critère	Description	Barème
Structure de Données Complexe	Utilisation pertinente des set/frozenset, defaultdict, tuples et match-case.	**4 pts**
Programmation Fonctionnelle	Maîtrise des fonctions map(), filter(), lambda et du passage de fonctions en argument.	**4 pts**
Moteur de Règles & Introspection	Bonne implémentation des *args, **kwargs, ainsi que des fonctions globals()/locals().	**3 pts**
Qualité du Code & Robustesse	Respect de la PEP8, typage, gestion propre des exceptions et de la clause else des boucles.	**3 pts**

LIVRABLES ATTENDUS

Un dépôt Git contenant :

1. L'arborescence du projet Python structurée en modules.
2. Le fichier requirements.txt
3. Un fichier main.py de démonstration permettant d'exécuter l'ensemble du pipeline sur un jeu de données de test fourni.
4. Un rapport synthétique (au format Jupyter Notebook .ipynb ou PDF) illustrant le fonctionnement.